

NAME - VISHAL MARANDI | ROLL - BTECH/10119/21 | DL ASSIGNMENT DATED 16th OCTOBER 2024

Implementing Logistic Regression with Autoencoder on MNIST DATASET

```
!pip install tensorflow
```

```
Requirement already satisfied: tensorflow in
/usr/local/lib/python3.10/dist-packages (2.17.0)
Requirement already satisfied: absl-py>=1.0.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (24.3.25)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1
in /usr/local/lib/python3.10/dist-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: h5py>=3.10.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (3.11.0)
Requirement already satisfied: libclang>=13.0.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: ml-dtypes<0.5.0,>=0.3.1 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (0.4.1)
Requirement already satisfied: opt-einsum>=2.3.2 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (24.1)
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!
=4.21.3,!4.21.4,!4.21.5,<5.0.0dev,>=3.20.3 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (3.20.3)
Requirement already satisfied: requests<3,>=2.21.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (2.32.3)
Requirement already satisfied: setuptools in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (71.0.4)
Requirement already satisfied: six>=1.12.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.16.0)
Requirement already satisfied: termcolor>=1.1.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (2.5.0)
Requirement already satisfied: typing-extensions>=3.6.6 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (4.12.2)
Requirement already satisfied: wrapt>=1.11.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.16.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.64.1)
Requirement already satisfied: tensorboard<2.18,>=2.17 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (2.17.0)
Requirement already satisfied: keras>=3.2.0 in
```

/usr/local/lib/python3.10/dist-packages (from tensorflow) (3.4.1)
Requirement already satisfied: tensorflow-io-gcs-filesystem<=0.23.1 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (0.37.1)
Requirement already satisfied: numpy<2.0.0,>=1.23.5 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.26.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
/usr/local/lib/python3.10/dist-packages (from astunparse>=1.6.0-
>tensorflow) (0.44.0)
Requirement already satisfied: rich in /usr/local/lib/python3.10/dist-
packages (from keras>=3.2.0->tensorflow) (13.9.2)
Requirement already satisfied: namex in
/usr/local/lib/python3.10/dist-packages (from keras>=3.2.0-
>tensorflow) (0.0.8)
Requirement already satisfied: optree in
/usr/local/lib/python3.10/dist-packages (from keras>=3.2.0-
>tensorflow) (0.13.0)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (2024.8.30)
Requirement already satisfied: markdown>=2.6.8 in
/usr/local/lib/python3.10/dist-packages (from tensorboard<2.18,>=2.17-
>tensorflow) (3.7)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0
in /usr/local/lib/python3.10/dist-packages (from
tensorboard<2.18,>=2.17->tensorflow) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in
/usr/local/lib/python3.10/dist-packages (from tensorboard<2.18,>=2.17-
>tensorflow) (3.0.4)
Requirement already satisfied: MarkupSafe>=2.1.1 in
/usr/local/lib/python3.10/dist-packages (from werkzeug>=1.0.1-
>tensorboard<2.18,>=2.17->tensorflow) (3.0.1)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.10/dist-packages (from rich->keras>=3.2.0-
>tensorflow) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.10/dist-packages (from rich->keras>=3.2.0-
>tensorflow) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.10/dist-packages (from markdown-it-py>=2.2.0-
>rich->keras>=3.2.0->tensorflow) (0.1.2)

```

#Importing Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras.layers import Input,Dense,Flatten,Reshape
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

#Loading the data
(x_train,y_train),(x_test,y_test)=mnist.load_data()

#Preprocessing the data
x_train=x_train.astype('float32')/255
x_test=x_test.astype('float32')/255

#Flattening
x_train_flatten=x_train.reshape(-1,28*28)
x_test_flatten=x_test.reshape(-1,28*28)

# 3. Build Autoencoder Model
input_img = Input(shape=(28 * 28,))
encoded = Dense(128, activation='relu')(input_img) # Change this to
128 features
decoded = Dense(28 * 28, activation='sigmoid')(encoded)
# Autoencoder model
autoencoder = Model(input_img, decoded)
# Encoder model to extract encoded features
encoder = Model(input_img, encoded)

# 4. Compile the Model
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
# 5. Train the Autoencoder
autoencoder.fit(x_train_flatten, x_train_flatten,
epochs=20,batch_size=256,shuffle=True,validation_data=(x_test_flatten,
x_test_flatten))
history=autoencoder.fit(x_train_flatten, x_train_flatten,
epochs=5,batch_size=256,shuffle=True,validation_data=(x_test_flatten,
x_test_flatten))

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# 6. Use Encoded Features for Logistic Regression

```

```

x_train_encoded = encoder.predict(x_train_flatten)
x_test_encoded = encoder.predict(x_test_flatten)
logistic = LogisticRegression(max_iter=1000)
logistic.fit(x_train_encoded, y_train)
# Predictions
y_pred = logistic.predict(x_test_encoded)
# 7. Evaluate and Analyze
accuracy = accuracy_score(y_test, y_pred)
print(f'Logistic Regression accuracy on encoded features: {accuracy *
100:.2f}%')

```

```

Epoch 1/20
235/235 ————— 6s 19ms/step - loss: 0.0656 - val_loss:
0.0654
Epoch 2/20
235/235 ————— 5s 19ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 3/20
235/235 ————— 7s 30ms/step - loss: 0.0655 - val_loss:
0.0654
Epoch 4/20
235/235 ————— 7s 29ms/step - loss: 0.0655 - val_loss:
0.0654
Epoch 5/20
235/235 ————— 10s 29ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 6/20
235/235 ————— 5s 23ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 7/20
235/235 ————— 11s 25ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 8/20
235/235 ————— 5s 23ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 9/20
235/235 ————— 8s 35ms/step - loss: 0.0655 - val_loss:
0.0654
Epoch 10/20
235/235 ————— 10s 32ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 11/20
235/235 ————— 7s 32ms/step - loss: 0.0656 - val_loss:
0.0654
Epoch 12/20
235/235 ————— 10s 31ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 13/20
235/235 ————— 8s 23ms/step - loss: 0.0653 - val_loss:
0.0654

```

```
Epoch 14/20
235/235 _____ 7s 29ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 15/20
235/235 _____ 4s 15ms/step - loss: 0.0654 - val_loss:
0.0653
Epoch 16/20
235/235 _____ 4s 17ms/step - loss: 0.0654 - val_loss:
0.0653
Epoch 17/20
235/235 _____ 9s 38ms/step - loss: 0.0655 - val_loss:
0.0653
Epoch 18/20
235/235 _____ 7s 31ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 19/20
235/235 _____ 6s 15ms/step - loss: 0.0653 - val_loss:
0.0653
Epoch 20/20
235/235 _____ 4s 15ms/step - loss: 0.0653 - val_loss:
0.0653
Epoch 1/5
235/235 _____ 5s 23ms/step - loss: 0.0654 - val_loss:
0.0653
Epoch 2/5
235/235 _____ 8s 15ms/step - loss: 0.0655 - val_loss:
0.0653
Epoch 3/5
235/235 _____ 7s 24ms/step - loss: 0.0653 - val_loss:
0.0653
Epoch 4/5
235/235 _____ 10s 24ms/step - loss: 0.0654 - val_loss:
0.0653
Epoch 5/5
235/235 _____ 7s 27ms/step - loss: 0.0656 - val_loss:
0.0654
```



1875/1875 ————— 4s 2ms/step

313/313 ————— 1s 2ms/step

Logistic Regression accuracy on encoded features: 92.24%

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469: ConvergenceWarning: lbfgs failed to converge (status=1):

STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
decoded_imgs=autoencoder.predict(x_test_flatten)
```

```
x_test_resaped=x_test_flatten.reshape(-1,28,28)
```

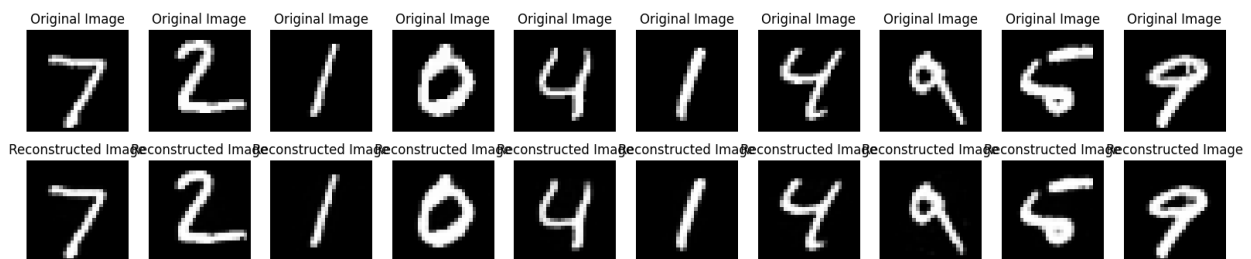
```
decoded_imgs=decoded_imgs.reshape(-1,28,28)
```

```
decoded_imgs_resaped=decoded_imgs.reshape(-1,28,28)
```

```
n=10
```

```
plt.figure(figsize=(20,4))
for i in range(n):
    ax=plt.subplot(2,n,i+1)
    plt.imshow(x_test_resaped[i],cmap='gray')
    plt.title("Original Image")
    plt.axis('off')
    ax=plt.subplot(2,n,i+1+n)
    plt.imshow(decoded_imgs_resaped[i],cmap='gray')
    plt.title("Reconstructed Image")
    plt.axis('off')
plt.show()
```

313/313 ————— 1s 3ms/step



1. Import Libraries

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt
```

2. Load and Preprocess the Dataset

```
(x_train, _), (x_test, _) = mnist.load_data()
```

Normalize the images to [0, 1]

```
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
```

Flatten the images (to pass through the fully connected autoencoder)

```
x_train_flatten = x_train.reshape(-1, 28 * 28)
x_test_flatten = x_test.reshape(-1, 28 * 28)
```

3. Add noise to the dataset

```
noise_factor = 0.75 # Change this to increase or decrease noise level
x_train_noisy = x_train_flatten + noise_factor *
np.random.normal(loc=0.0, scale=1.0, size=x_train_flatten.shape)
x_test_noisy = x_test_flatten + noise_factor *
np.random.normal(loc=0.0, scale=1.0, size=x_test_flatten.shape)
```

Clip the values to be between 0 and 1 (valid pixel range)

```

x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# 4. Build Autoencoder Model
input_img = Input(shape=(28 * 28,))
encoded = Dense(128, activation='relu')(input_img)
decoded = Dense(28 * 28, activation='sigmoid')(encoded)

# Autoencoder model
autoencoder = Model(input_img, decoded)

# 5. Compile the Model
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# 6. Train the Autoencoder
history = autoencoder.fit(x_train_noisy, x_train_flatten,
                          epochs=5,
                          batch_size=256,
                          shuffle=True,
                          validation_data=(x_test_noisy,
x_test_flatten))

# 7. Predict Denoised Images
denoised_imgs = autoencoder.predict(x_test_noisy)

# Reshape the noisy and denoised images back to 28x28 for
visualization
x_test_noisy_resaped = x_test_noisy.reshape(-1, 28, 28)
denoised_imgs_resaped = denoised_imgs.reshape(-1, 28, 28)

# 8. Visualize Original Noisy and Denoised Images
n = 10 # number of images to display
plt.figure(figsize=(20, 6))
for i in range(n):
    # Display noisy images
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test_noisy_resaped[i], cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    # Display denoised images
    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(denoised_imgs_resaped[i], cmap='gray')
    plt.title("Denoised")
    plt.axis('off')

    # Display original images for comparison
    ax = plt.subplot(3, n, i + 1 + 2 * n)
    plt.imshow(x_test[i], cmap='gray')
    plt.title("Original")

```



```
plt.axis('off')  
plt.show()
```

Epoch 1/5
235/235 ————— 5s 16ms/step - loss: 0.3215 - val_loss: 0.1912
Epoch 2/5
235/235 ————— 5s 16ms/step - loss: 0.1845 - val_loss: 0.1669
Epoch 3/5
235/235 ————— 5s 16ms/step - loss: 0.1648 - val_loss: 0.1568
Epoch 4/5
235/235 ————— 5s 15ms/step - loss: 0.1556 - val_loss: 0.1513
Epoch 5/5
235/235 ————— 5s 20ms/step - loss: 0.1504 - val_loss: 0.1479
313/313 ————— 1s 3ms/step

